

Coursera

UNIVERSITY OF TORONTO



AI Agents and Agentic AI with Python & Generative AI

Module 1

Agentic AI Concepts

Module 2

AI Agents, Tools, Actions, & Language

Module 3

GAME, A Conceptual Framework for AI Agents

Module 4

Agent Tool Management

Module 5

Refining Agent Tools Up to Date with the Age of AI Agents

Keeping Agent Tools Up to Date with Python Decorators

Ungraded Plug-in 12-min

Tool Organization for Agents

Ungraded Plug-in 12-min

Reflexioning Our AGENT Agent

Ungraded Plug-in 15-min

Try Out the README Agent with the Decorator

Reading 10-min

Module 5

Refining Agent Tools Up to Date with the Age of AI Agents

Build the Impossible with AI Agents

Video 1 min

Reflexioning How We Teach Innovation

Video 4-min

The Inventory Management Agent

Ungraded Plug-in 15-min

Introducing a New Form of Computing

Video 7 min

New Ways to Access and Extract Information

Video 3 min

Keeping Agent Tools Up to Date with Python Decorators

Tool Decorators: Keeping Code and Documentation in Sync

When building AI agents, one of the most challenging aspects is maintaining accurate documentation about the tools our agent can use. Every time we modify a tool's parameters or behavior, we need to update its description and parameter schema. If these get out of sync, our agent might try to use tools incorrectly, leading to failures that can be hard to debug. Let's explore how we can solve this problem elegantly using Python decorators.

The Challenge of Tool Documentation

Imagine we're writing a tool for our agent to read files. Without any special handling, we might write something like this:

```
# In our action registry setup
action_registry.register(action=
    name="read_file",
    function=read_file,
    description="Reads content from a specified file",
    parameters={
        "type": "object",
        "properties": {
            "file_path": {"type": "string"}
        },
        "required": ["file_path"]
    }
))

# The actual function
def read_file(file_path: str) -> str:
    """Reads and returns the content of a file."""
    with open(file_path, "r") as f:
        return f.read()
```

This approach has several problems. If we add a new parameter to `read_file`, we need to remember to update the parameters schema. If we change the function's behavior, we need to update the description. It's easy for these to become inconsistent over time.

The Power of Decorators

Instead, we can use Python's decorator system to automatically extract and maintain this information. Here's how it works:

```
@register_tool(tags=["file_operations"])
def read_file(file_path: str) -> str:
    """Reads and returns the content of a file from the specified path.

    The function opens the file in read mode and returns its entire contents
    as a string. If the file doesn't exist or can't be read, it raises an
    appropriate exception.

    Args:
        file_path: The path to the file to read

    Returns:
        The contents of the file as a string
    """
    with open(file_path, "r") as f:
        return f.read()
```

Our decorator examines the function and automatically:

1. Uses the function name as the tool name
2. Extracts the docstring for the description
3. Analyzes type hints and parameters to build the schema
4. Registers the tool in a central registry

Implementing the Decorator

Let's look at how this magic happens by first understanding the decorator and then examining the helper function that does the heavy lifting.

```
def register_tool(tool_name=None, description=None,
                  parameters_override=None, terminal=False, tags=None):
    """Registers a function as an agent tool."""
    def decorator(func):
        # Extract all metadata from the function
        metadata = get_tool_metadata(
            func=func,
            tool_name=tool_name,
            description=description,
            parameters_override=parameters_override,
            terminal=terminal,
            tags=tags
        )

        # Register in our global tools dictionary
        tools_metadata["tool_name"] = {
            "description": metadata["description"],
            "parameters": metadata["parameters"],
            "function": metadata["function"],
            "terminal": metadata["terminal"],
            "tags": metadata["tags"]
        }

        # Also maintain a tag-based index
        for tag in metadata["tags"]:
            if tag not in tools_by_tag:
                tools_by_tag[tag] = []
            tools_by_tag[tag].append(metadata["tool_name"])

        return func
    return decorator
```

This `register_tool` decorator is designed to transform regular Python functions into callable tools for an agent system. Here's what's happening in detail:

1. The decorator accepts several optional parameters that configure how the function will be registered as a tool.
2. It returns an inner function (`decorator`) that takes the actual function being decorated (`func`) as its parameter.
3. Within this inner function, it first calls `get_tool_metadata()` to analyze and extract all relevant information from the function to turn it into a tool description that the agent can use.
4. It then registers the tool in a global dictionary called `tools`, using the tool name as a key. The `AgentRegistry` can be configured to lookup tools here.
5. For improved discoverability, it also maintains a tag-based index in `tools_by_tag`, allowing tools to be looked up by category. This tagging allows us to define sets of tools that should be used together.
6. Finally, it returns the original function unchanged, allowing it to still be called normally while also being available as a tool.

This pattern creates a clean separation between the definition of tools and their registration, making the codebase much more maintainable.

Now, let's examine the helper function that does the detailed analysis:

```
def get_tool_metadata(func, tool_name=None, description=None,
                     parameters_override=None, terminal=False, tags=None):
    """Extracts metadata for a function to use in tool registration."""

    # Use function name if no tool_name provided
    tool_name = tool_name or func.__name__

    # Use docstring if no description provided
    description = description or (func.__doc__ if func.__doc__ else "No description
provided.")

    # If no parameter override, analyze the function
    if parameters_override is None:
        signature = inspect.signature(func)
        type_hints = get_type_hints(func)

        # Build JSON schema for arguments
        args_schema = {
            "type": "object",
            "properties": {},
            "required": []
        }

        # Examine each parameter
        for param_name, param in signature.parameters.items():
            # Skip special parameters
            if param_name in ["action_context", "action_agent"]:
                continue

            # Convert Python types to JSON schema types
            param_type = type_hints.get(param_name, str)
            param_schema = {
                "type": get_json_type(param_type)
            }

            args_schema["properties"][param_name] = param_schema

            # If parameter has no default, it's required
            if param.default == inspect.Parameter.empty:
                args_schema["required"].append(param_name)

        else:
            args_schema = parameters_override

    return {
        "tool_name": tool_name,
        "description": description,
        "parameters": args_schema,
        "function": func,
        "terminal": terminal,
        "tags": tags or []
    }
```

This helper function performs introspection on the decorated function to automatically extract the metadata needed to describe the tool to the agent:

1. It determines the tool's name, defaulting to the function's name if not explicitly provided.
2. It extracts a description from the function's docstring if one isn't explicitly provided.
3. For parameter handling, it does sophisticated introspection using Python's `inspect` and `typing` modules:
 - It captures the function's signature to identify all parameters
 - It extracts type annotations using `get_type_hints()`
 - It builds a JSON schema that describes the expected input structure
 - It identifies required parameters (those without default values)
 - It intelligently skips special context parameters like `action_context` and `action_agent`
 - It converts Python types to JSON schema types via a helper function `get_json_type()`
4. It packages all this metadata into a dictionary that the decorator can use to register the tool.

This thorough introspection enables tools to be defined with minimal boilerplate while providing rich metadata for the agent system to understand how to call each tool properly. The tool description will always match the function's signature and docstring, ensuring that the agent has the most accurate information available.

Why Create a Decorator?

You will see this decorator approach in many different agent frameworks. Here is why:

1. **Single Source of Truth:** The function itself becomes the authoritative source for all tool information. The decorator describes what it does, the type hints define its parameters, and the implementation shows how it works.
2. **Automatic Updates:** When we modify the function's signature or documentation, the tool registration automatically stays in sync. No more hunting through code to update parameter schemas.
3. **Better Organization:** The tag system allows us to categorize tools and find related functionality. We can easily get all "file_operations" tools or all "database_tools".
4. **Improved Development Experience:** We write our tools as normal Python functions with standard documentation. The decorator handles all the complexity of making them available to our agent.

Consider how this simplifies adding a new parameter:

```
@register_tool(tags=["file_operations"])
def read_file(file_path: str, encoding: str = "utf-8") -> str:
    """Reads and returns the content of a file.

    Args:
        file_path: The path to the file to read
        encoding: The character encoding to use (default: utf-8)
    """
    with open(file_path, "r", encoding=encoding) as f:
        return f.read()
```

The tool's parameter schema automatically updates to include the new `encoding` parameter as optional (since it has a default value). We didn't need to manually update any registration code or documentation.

Expand

Completed

Go to next item

Like

Dislike

Report as issue